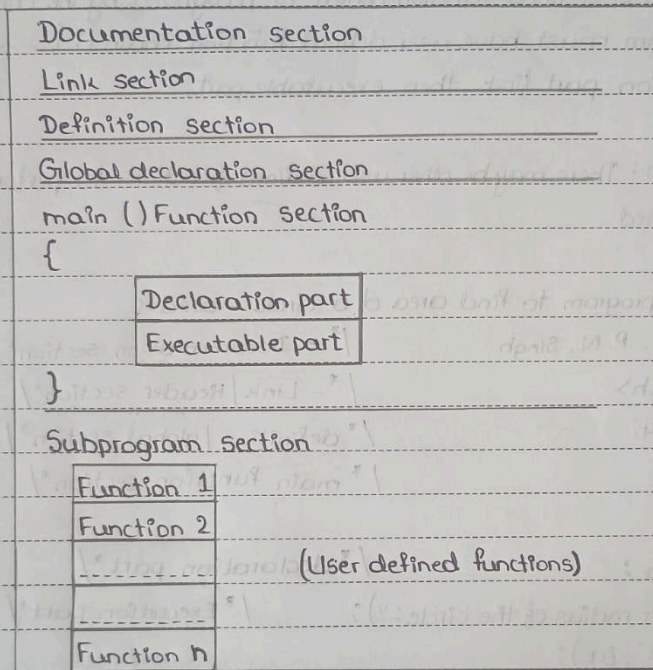


Solution-Model Answer

Prof. Rajni tiwari, Prof.Reshma, Prof. Dhyviya, Prof. Saumya, Prof. kiran,Dr Sivasakti, Prof Shilpashree										
Model Question Paper- I – January 2025										
Sub	Programming in C					Sub code	1BRIT105/ 205	Branch	ISE, AIDS, AIML, CSML	
Date	.01.2025	Duration	180 mins	Max Marks	100	Sem /Sec	I Sem P-Cycle (A- H)	OBE		
Module - 1								M	L	C
Qu1.	a)Define data type. Explain primitive data types supported in C language with Example. Ans:A data type in C defines the type of data a variable can store, the amount of memory it occupies, and the operations that can be performed on that data. The primitive data types supported in C are ● int: It is used to store whole numbers. Memory size: 2 or 4 bytes (depends on system) <i>Example:</i> int age = 20; ● float: is a primitive data type in C that is used to store real numbers (numbers with decimal or fractional values) with single precision. Memory size: 4 bytes <i>Example:</i> float salary = 25000.75; ● double: It is used to store decimal numbers with double precision. Memory size: 8 bytes <i>Example:</i> double pi = 3.14159; ● char: It is used to store a single character. Memory size: 1 byte <i>Example:</i> char grade = 'A';						[6]	L2	CO1	
	b)Explain the general form of a C program with examples. Ans:						8	L2	CO1	

1) Explain the basic structure of a C program with example.



• Documentation Section: This section is used to write problem, file name, developer, date etc in comment lines within /*...*/ or separate line comment may start with //. Compiler ignores this section. Documentation enhances the readability of a program.

• Link section: To include header and library files whose in-built structures are to be used. Linker also requires these files to build a program executable. Files are included with directive, #include

- Definition section : To define macros and symbolic constants by preprocessor directive #define.
- Global section : to declare global variables - to be accessed by all functions.
- main() is the user defined function which is recognized by the compiler first. So, all C program must have user defined function main() { }. It should have declaration part first then executable part.
- Sub program section : There may be other user defined functions to perform specific task when called.

```

/* Example : a program to find area of a circle - area.c
Dr. P. N. Singh                                /* - Documentation section */
#include <stdio.h>                                /* - Link / Header section */
#define PI 3.14                                  /* definition / global section */
int main()                                       /* main function section */
{
    float r, area;                               /* declaration part */
    printf("Enter radius of the circle:");      /* Execution part */
    scanf("%f", &r);
    area = PI * r * r; /* using symbolic constant PI */
    printf("Area of circle = %0.3f square unit \n", area);
    return (0);
}

```

c) Develop a C program to multiply, subtract and division by taking two whole numbers. Choose suitable datatypes for variables.

Ans:

```
#include <stdio.h>
```

```
int main() {
```

```
    int num1, num2;    // Whole numbers (integers)
```

```
    int multiplication, subtraction;
```

```
    float division;    // Division result may not always be an integer
```

[6]

L3

CO5

	<pre> // Taking input from the user printf("Enter first whole number: "); scanf("%d", &num1); printf("Enter second whole number: "); scanf("%d", &num2); // Calculations multiplication = num1 * num2; subtraction = num1 - num2; // Check for division validity (to avoid division by zero) if (num2 != 0) { division = (float)num1 / num2; printf("\nDivision: %d / %d = %.2f\n", num1, num2, division); } else { printf("\nDivision not possible (division by zero).\n"); } // Display results printf("Multiplication: %d * %d = %d\n", num1, num2, multiplication); printf("Subtraction: %d - %d = %d\n", num1, num2, subtraction); return 0; } </pre>			
2	a)What is variable? Explain the rules to construct variables. Classify the following as valid/invalid Identifiers. i) num2 ii) \$num1 iii) +add iv) a_2 v) 199_space vi) _apple vii) #12 A variable in C programming is a named memory location used to store data.It can hold values that may change during the program execution. Syntax : <i>type variable_name/variable_list</i>; Here, <i>type</i> must be a valid data type , <i>variable_name</i> is the valid name and <i>variable_list</i> may consist of one or more identifier names separated by commas. Here are some declarations: <pre> int i, j, l; short int si; </pre>	[6]	L2	CO1

unsigned int ui;

double balance, profit, loss;

Example: int age = 25;

Here, age is a variable that stores an integer value **25**.

Rules to Construct Variables (Identifiers)

When creating a variable name in C:

1. **Can use letters (A–Z, a–z), digits (0–9), and underscore (_)** : A variable name in C can contain letters, digits, and the underscore symbol.
2. **Must begin with a letter or underscore** : It must always start with a letter or an underscore but should never begin with a digit.
(Cannot start with a number)
3. **No spaces or special characters** : Spaces or special characters such as +, -, *, /, \$, and # are not allowed in variable names
4. **Cannot use reserved keywords** : Variable names should not be the same as reserved keywords like int, return, for, or if.
5. **Case sensitive** : They are also case-sensitive, which means names like age and Age are treated as different identifiers.
6. **Variable names should be meaningful** : variable names should be meaningful so that the program is easy to read and understand.

Classification of Identifiers

Identifier	Valid /Invalid	Reason
num2	Valid	Starts with a letter and contains only letters & digits
\$num1	Invalid	\$ is not allowed in C identifiers
+add	Invalid	Cannot start with + or contain operators
a_2	valid	Letters, digits, and _ allowed
199_space	invalid	Cannot start with a digit
_apple	Valid	Can start with an underscore
#12	Invalid	# is not allowed in identifiers

	b)Show the evaluation of the following expressions with intermediate and final values. i) $x = a - b/3 + c * 2 - 1$ when $a = 9$, $b = 12$, $c = 3$ ii) $10! = 10 \parallel 5 < 4 \&\& 8$. A. $x = a - b/3 + c * 2 - 1$ when $a = 9$, $b = 12$, $c = 3$ Step-by-step Evaluation $b/3 = 12/3 = 4$ $c * 2 = 3 * 2 = 6$ - Substitute the values: $x = 9 - 4 + 6 - 1$ $= 5 + 6 - 1$ $= 11 - 1 = 10$ b. $10! = 10 \parallel 5 < 4 \&\& 8$. Evaluate left side: $10! \rightarrow !10 \rightarrow 0$ Left side: 0 Evaluate right side using precedence Operators order: $<$ $\&\&$ $\parallel$ $10 \parallel 5 < 4 \&\& 8$ $10 \parallel 0 \&\& 8$ $10 \parallel 0$ 1 Final Evaluation : $10! = 10 \parallel 5 < 4 \&\& 8.$ $0 = 1$ 0	[8]	L2	CO1
	c)Develop C program which takes as input p, t, r and calculates the simple interest. Choose suitable data types for variables. Ans: Program to calculate Simple Interest (SI) using the formula:	6	L3	CO5

$$\text{Simple Interest} = \frac{P \times T \times R}{100}$$

Code:

```
#include <stdio.h>

int main() {
    float p, t, r;
    float si;
    printf("Enter Principal Amount (P): ");
    scanf("%f", &p);

    printf("Enter Time in years (T): ");
    scanf("%f", &t);

    printf("Enter Rate of Interest (R): ");
    scanf("%f", &r);

    // Calculation of Simple Interest
    si = (p * t * r) / 100;

    // Output result
    printf("Simple Interest = %.2f\n", si);

    return 0;
}
```

3	a)With a suitable example, explain formatted input and output statements. Ans: In C language, formatted input and output statements allow the user to read and display data in a controlled, structured, and formatted manner. The two most commonly used functions are printf() for output and scanf() for input. Both these functions use format specifiers to define the type and format of data. 1. Formatted Output Statement: printf() printf() is used to display values, text, and results on the screen in a specified format. Features ● It uses format specifiers such as %d (int), %f (float), %c (char), %s (string). ● Formatting options like number of decimal places, alignment, width etc. can be controlled. Example: <pre>#include<stdio.h> Void main(){ int age = 20; float marks = 85.75; printf("Age = %d\n", age); printf("Marks = %.2f\n", marks);}</pre>	6	L2	CO1
---	---	---	----	-----

2. Formatted Input Statement: scanf()

scanf() is used to take input from the user in a specific format.

Features

- Reads data from keyboard based on format specifiers.
- The **address operator (&)** is used to store the value in variables.

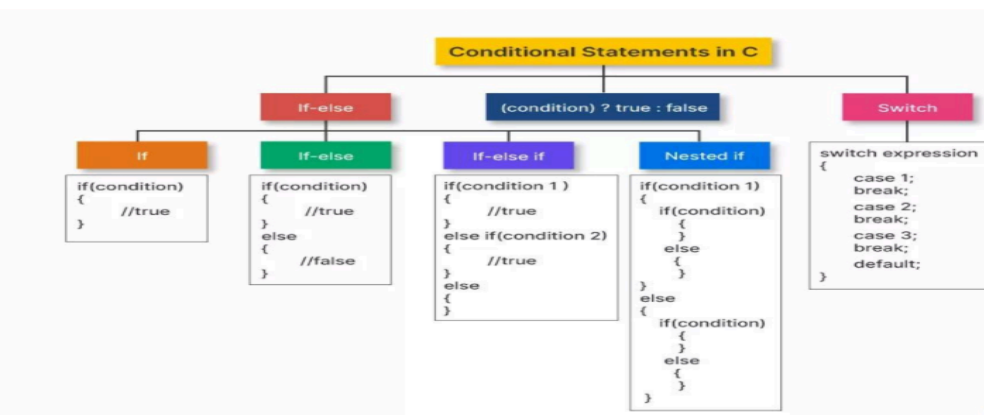
Example:

```
#include<stdio.h>
Void main(){
int a;
float b;
printf("Enter an integer and a float: ");
scanf("%d %f", &a, &b);
}
```

b) List the conditional branching statements in 'C'. Explain any two with suitable Examples.

Ans:

conditional branching statements are used to control the flow of execution based on certain conditions. Here are two commonly used conditional branching Statements:



1. if Statement

The if statement is used to execute a block of code only if a specified condition is true. If the condition is false, the code block is skipped.

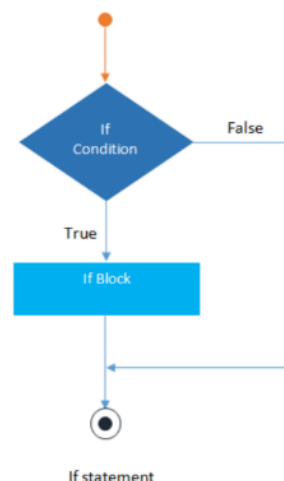
Syntax:

1. Simple if statement:

The general form of single if statement is :

```
if ( test expression)
{
statement-Block;
}
statement-Blocks;
```

Flowchart:



Working:

Condition: This is a Boolean expression that evaluates to either true (non-zero value) or false (zero). If the condition is true, the statements inside the block are executed.

Statement block: This is the code that runs if the condition is true. If the condition is false, this block is skipped.

Example:

```
#include<stdio.h>
int main()
{
    int num;
    num = 4;
    if(num%2 == 0)
    {
        printf("\n%d is even",num);
    }
    printf("\nOut of if body...");
    return 0;
}
```

Output:

4 is even

Out of if body...

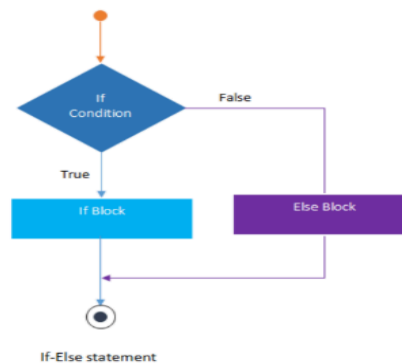
2. if else statement

The if-else statement is a two way decision making statement in C. The if-else statement allows two different blocks of code to execute based on whether a condition is true or false. If the condition is true, the if block is executed; otherwise, the else block is executed.

Syntax:**Flowchart:**

The general form of if-else statement is

```
if ( test expression)
{
    statement-block1;
}
else
{
    statement-block2;
}
```

**Working:**

Condition: This is a Boolean expression that evaluates to either true (non-zero value) or false (zero). If the condition is true, the statements inside the if block are executed. If the condition is

false else block will execute.

Example:

```
#include<stdio.h>
int main()
{
    int num;
    num = 4;
    if(num%2 == 0)
    {
        printf("\n%d is even",num);
    }
}
```

```

}
else
{
printf("\n%d is odd",num);
}
printf("\nOut of if-else...");
return 0;
}
Output 1: for num=4
4 is even
Out of if-else...
Output 2: for num=5
5 is odd
Out of if-else...

```

3c) Develop a C program to print Floyd's triangle for N rows (N > 0). Choose suitable control statements.

[for n=4]

```

1
2 3
4 5 6
7 8 9 10

```

Ans:

```
#include <stdio.h>
```

```

int main() {
    int n, i, j;
    int num = 1;    // starting number

    printf("Enter number of rows (N > 0): ");
    scanf("%d", &n);
    // Outer loop for rows
    for (i = 1; i <= n; i++) {
        // Inner loop for printing numbers in each row
        for (j = 1; j <= i; j++) {
            printf("%d ", num);
            num++;    // increment the number
        }
        printf("\n");
    }

    return 0;
}

```

Output:

```
Enter number of rows (N > 0): 4
```

```

1
2 3
4 5 6
7 8 9 10

```

```
PS C:\Users\91819\OneDrive\Desktop\C_Programming>
```

8

L3

CO5

4	a) Explain Jump Statements, Expression Statements, Block Statements with suitable examples. Ans: In C language, statements are the smallest executable units. They control program flow, perform operations, or group multiple statements together. Three important types are Jump Statements, Expression Statements, and Block Statements. 1. Jump Statements Jump statements are used to transfer control from one part of the program to another. They help in breaking loops, skipping code, or returning from functions. Types of Jump Statements • break • continue • goto • return Example of break: <pre>for (int i = 1; i <= 5; i++) { if (i == 3) break; // exits the loop when i = 3 printf("%d ", i); }</pre> Example of continue: <pre>for (int i = 1; i <= 5; i++) { if (i == 3) continue; // skips printing 3 printf("%d ", i); }</pre> Example of goto: <pre>#include <stdio.h> int main() { int num = 5; if (num > 0) goto positive; // jumps to the label printf("This line will be skipped.\n"); positive: // label printf("Number is positive.\n"); return 0; }</pre> Example of return: <pre>#include <stdio.h> int display() { printf("Inside display function.\n"); return 0; // returns control to main } int main() { printf("Before calling display.\n"); display(); // function call printf("After calling display.\n"); }</pre>	6	L2	CO1
---	--	---	----	-----

```
    return 0;    // terminates the main function
}
```

2. Expression Statements

An expression statement consists of an expression followed by a semicolon. It performs operations like assignments, arithmetic, function calls, etc.

Example

```
int a = 5, b = 3;
a = a + b;    // expression statement
printf("%d", a);    // function call expression statement
```

3. Block Statements

A block statement (also called a compound statement) is a group of statements enclosed within { }. Blocks are treated as a single statement and used in loops, functions, and conditional structures.

Example

```
if (x > 0) {
    printf("Positive\n");
    x = x + 10;
}
```

b) Explain the role of break and continue statements in C with suitable examples.

The break statement is used to terminate the execution of the nearest enclosing loop in which it appears.

Syntax : break;

```
#include<stdio.h>
int main() {
    int i=0;
    while(i<=10) {
        if(i==5)
            break;
        printf("\n %d",i);

        i=i+1;
    }
    return 0;
}
```

Output
0 1 2 3 4

The continue statement can only appear in the body of a loop. When the compiler encounters a continue statement then the rest of the statements in the loop are skipped and the control is unconditionally transferred to the loop-continuation portion of the nearest enclosing loop.

[6]

L2

CO2

	<pre> syntax continue; #include<stdio.h> int main() { int i; for(i=0;i<=10;i++) { if(i==5) continue; printf("\n %d",i); } return 0; } Output 0 1 2 3 4 6 7 8 9 10 </pre> c)Develop a simple calculator program in C language for simple operations like addition, subtraction, multiplication and division. Choose suitable selection Statement. <pre> /* calculator by switch */ int main() { float n1,n2,result; int choice; printf("Enter 2 numbers : "); scanf("%f%f",&n1, &n2); printf("1. Addition 2. Subtraction 3. Multiplication 4. Division:\n"); printf("Enter your choice number : "); scanf("%d", &choice); switch(choice) </pre>			
		[8]	L3	CO5

```

{
case 1 : result = n1+n2;

break;
case 2 : result = n1-n2;

break;
case 3 : result = n1*n2;

break;
case 4 : if(n2 == 0)
{ printf("divide error!!!\n");
return (0); }
else
result = n1/n2;

break;
default: printf("Wrong choice!\n");
}

printf("Result = %f\n",result);
return (0);
}

```

Expected Output:

Enter 2 numbers : 50 25

1. Addition 2. Subtraction 3. Multiplication 4. Division:

Enter your choice number : 4

Result = 2.000000

5	a)Define an array. How are single dimension and two-dimensional arrays declared and initialized? Illustrate with suitable examples. Ans: An array is a collection of similar data items stored at contiguous memory locations and elements can be accessed randomly using indices of an array. (1) 1-D Array declaration by specifying size Syntax: datatype array[size]; // Array declaration by specifying size int arr1[10]; (2) 1-D Array declaration by initializing elements // Array declaration by initializing elements int arr[] = { 10, 20, 30, 40 }; Compiler creates an array of size 4. above is same as int arr[4] = {10, 20, 30, 40} // Program to take 5 values from the user and store them in an // print the elements stored in the array #include <stdio.h> <pre>int main() { int values[10] ; int i; printf("Enter 5 integers: "); // 2 4 3 5 7 // taking input and storing it in an array for(i = 0; i < 5; i++) { scanf("%d", &values[i]); } // Reading 2 4 3 5 7 } printf("Displaying integers: "); // printing elements of an array for(i = 0; i < 5; i++) { printf("%d\n", values[i]); } return 0; }</pre> 2D array Syntax to declare 2D array data_type array_name[rows][columns]; example	[8]	L2	CO2
---	---	-----	----	-----

<pre>int matrix[3][4]; // 3 rows, 4 columns</pre> Initialization of Two-Dimensional Arrays <pre>int matrix[2][3] = { {1, 2, 3}, {4, 5, 6} };</pre> Example Program Using 2D Array <pre>#include <stdio.h> int main() { int matrix[2][3] = { {10, 20, 30}, {40, 50, 60} }; printf("Matrix elements:\n"); for (int i = 0; i < 2; i++) { for (int j = 0; j < 3; j++) { printf("%d ", matrix[i][j]); } printf("\n"); } return 0; }</pre>	6	L2	CO2
b) Define variable length array. Illustrate how variable length array is different from static array Ans: A Variable Length Array is a data structure introduced in C99 that allows the length of an array to be specified using a variable whose value is known at runtime, enabling more flexible memory use on the stack. <pre>#include <stdio.h> int main() { int n; printf("Enter the size of the array: "); scanf("%d", &n); // User decides the size at runtime int arr[n]; // Variable Length Array (VLA) // Take input into the array printf("Enter %d numbers:\n", n); for (int i = 0; i < n; i++) {</pre>	[6]	L2	CO2

```

scanf("%d", &arr[i]);

}

// Print the array

printf("You entered: ");

for (int i = 0; i < n; i++) {

printf("%d ", arr[i]);

}

printf("\n");

return 0;

}

```

OUTPUT:

Enter the size of the array: 4

Enter 4 numbers:

10 20 30 40

You entered: 10 20 30 40

A **static array** is an array with a size that is known before the program starts executing, and memory for the array is allocated at compile time.

```
#include <stdio.h>
```

```
int main() {
```

```
    int numbers[5] = {10, 20, 30, 40, 50}; // Static array with fixed size 5
```

```
    for(int i = 0; i < 5; i++) {
```

```
        printf("%d ", numbers[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

- A **static array** has a fixed size known at compile time.
- A **variable length array** (VLA) has a size determined at runtime.

5.c	Develop a C program to swap the values of two integer variables using pointers. Solution: <pre>#include <stdio.h> void swap(int *a, int *b) { int temp = *a; *a = *b; *b = temp; } void main() { int balance1, balance2; printf("Enter balance for Account 1: "); scanf("%d", &balance1); printf("Enter balance for Account 2: "); scanf("%d", &balance2); printf("\nBefore Swapping:\n"); printf("Account 1 Balance: %d\n", balance1); printf("Account 2 Balance: %d\n", balance2); swap(&balance1, &balance2); printf("\nAfter Swapping:\n"); printf("Account 1 Balance: %d\n", balance1); printf("Account 2 Balance: %d\n", balance2); }</pre>	[6]	L3	CO2
6a.	Define a pointer. How do you declare and initialize pointers in C. Explain accessing array elements using a pointer. Pointer A pointer is a variable that stores the memory address of another variable. • A normal variable stores a value. Example: int x = 10; → stores 10 • A pointer stores the address of a variable. Example: int *p = &x; → stores address of x So, pointer = “address-holding variable”. Declaration of Pointers in C Syntax <pre>data_type *pointer_name;</pre> Examples <pre>int *p; // pointer to an integer float *fp; // pointer to a float</pre>	[8]	L1	CO2

```
char *cp;    // pointer to a char
```

Initialization of Pointers

Pointers must be assigned with **addresses** using the address-of operator &.

Example

```
int x = 10;
```

```
int *p = &x;    // p stores the address of x
```

Safe Initialization

```
int *p = NULL;  // meaning: pointer is not pointing anywhere
```

Accessing Value Using Pointer (Dereferencing)

To get the value stored at the address inside a pointer, we use * (dereference operator):

```
int x = 10;
```

```
int *p = &x;
```

```
printf("%d", *p); // prints 10
```

Accessing Array Elements Using a Pointer

In C, the **name of the array acts as a constant pointer** to the first element.

```
int arr[5] = {10, 20, 30, 40, 50};
```

```
int *p = arr;
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int arr[5] = {10, 20, 30, 40, 50};
```

```
    int *p = arr;    // p points to arr[0]
```

```
    printf("arr[0] = %d\n", *p);    // 10
```

```
    printf("arr[1] = %d\n", *(p + 1)); // 20
```

```
    printf("arr[2] = %d\n", *(p + 2)); // 30
```

```
    printf("arr[3] = %d\n", *(p + 3)); // 40
```

```
    printf("arr[4] = %d\n", *(p + 4)); // 50
```

```
    return 0;
```

```
}
```

6b

Show with a suitable program, how a single dimensional array can be passed to a function Passing a One-Dimensional Array to a Function .

[6]

L2

CO2

Ans:
In C, a one-dimensional array can be passed to a function by passing the **base address** (address of the first element) of the array.
When an array is passed to a function, the function receives a **pointer** to the first element, and it can access all elements using this pointer.

```
#include <stdio.h>
```

```
// Function prototype: arr[] or *arr both work
```

```
void display(int arr[], int size);
```

```
int main() {
```

```
    int numbers[5] = {10, 20, 30, 40, 50};
```

```
    // Passing array to the function
```

```
    display(numbers, 5);
```

```
    return 0;
```

```
}
```

```
// Function definition
```

```
void display(int arr[], int size) {
```

```
    int i;
```

```
    printf("Array elements are:\n");
```

```
    for (i = 0; i < size; i++) {
```

```
        printf("%d ", arr[i]);
```

```
    }
```

```
}
```

A single-dimensional array is passed to a function by passing its base address. Inside the function, the array is accessed using the pointer parameter.

6c.

Ans :

Develop a C program that reads N integers, stores them in an array and calculates the sum of all array elements.

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, sum = 0;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    int arr[n]; // Declare array of size n
```

[6]

L3

CO2

7a	<pre> printf("Enter %d integers:\n", n); for (i = 0; i < n; i++) { scanf("%d", &arr[i]); // Read each element } // Calculate sum for (i = 0; i < n; i++) { sum += arr[i]; } printf("Sum of array elements = %d\n", sum); return 0; } </pre> Define function. Explain the syntax of function definition and function declaration with a simple example. Functions are the building blocks of C and the place where all program activity occurs. The General Form of a Function: <pre> ret-type function-name(parameter list) { body of the function } </pre> A function declaration tells the compiler: • the name of the function • the return type • the parameter types It does not contain the body (the actual code). It ends with a semicolon. Syntax: <code>return_type function_name(parameter_list);</code> Example: <code>int add(int a, int b); // function declaration</code> <pre> int add(int a, int b) // function definition { return a + b; } #include <stdio.h> int add(int a, int b); // function declaration int main() { </pre>	[6]	L2	CO3
-----------	---	------------	-----------	------------

```

int result = add(3, 5);

printf("Sum = %d", result);

return 0;
}

int add(int a, int b) { // function definition

    return a + b;
}

OutPut: Sum = 8

```

Define dynamic memory allocation. List and explain the different functions to handle dynamic memory allocation in C.

Dynamic memory allocation refers to the process of allocating memory at runtime (when the program is running) as opposed to static memory allocation, where memory is allocated before execution starts (usually at compile time).

Dynamic memory allocation allows programs to be more flexible by using memory as needed and releasing it when it's no longer required, thus avoiding memory waste and ensuring efficient memory use.

In C, dynamic memory allocation is handled through a set of library functions provided by the `stdlib.h` library.

C provides several functions to allocate, deallocate, and manipulate dynamic memory. The functions are:

1. `malloc()` (Memory Allocation)

- **Purpose:** Allocates a block of memory of a specified size and returns a pointer to it.

Syntax:

```
void* malloc(size_t size);
```

- **Description:** The `malloc()` function allocates a specified number of bytes of memory and returns a pointer to the first byte of the allocated memory. The initial content of the memory block is **not initialized** (may contain garbage values).
 - If the memory allocation fails, `malloc()` returns `NULL`.

Example:

```

int *arr = (int*) malloc(5 * sizeof(int)); // Allocates
memory for 5 integers

if (arr == NULL) {

```

[6]

L2

CO3

7b.

```
printf("Memory allocation failed.\n");
```

```
}
```

2. `calloc()` (Contiguous Allocation)

- **Purpose:** Allocates memory for an array of elements and initializes all the elements to zero.

Syntax:

```
void* calloc(size_t num_elements, size_t  
size_of_element);
```

- **Description:** `calloc()` allocates memory for an array of `num_elements`, each of size `size_of_element`. Unlike `malloc()`, `calloc()` also initializes the allocated memory to **zero**.
 - If the memory allocation fails, `calloc()` returns `NULL`.

Example:

```
int *arr = (int*) calloc(5, sizeof(int)); // Allocates  
memory for 5 integers and initializes to 0
```

```
if (arr == NULL) {  
    printf("Memory allocation failed.\n");  
}
```

3. `realloc()` (Reallocate Memory)

- **Purpose:** Changes the size of a previously allocated memory block.

Syntax:

```
void* realloc(void* ptr, size_t new_size);
```

- **Description:** `realloc()` is used to **resize** a previously allocated memory block (pointed to by `ptr`). It can increase or decrease the size of the block.
 - If `realloc()` successfully resizes the memory block, it returns a pointer to the newly allocated memory. If it fails, it returns `NULL`, and the original memory is **not freed**.
 - If the new size is larger, the new memory is not initialized; the extra memory may contain garbage values.

Example:

```
int *arr = (int*) malloc(5 * sizeof(int)); // Initially
```



```
allocate memory for 5 integers

// Resize the allocated memory to hold 10 integers
arr = (int*) realloc(arr, 10 * sizeof(int));
if (arr == NULL) {
    printf("Memory reallocation failed.\n");
}
```

4. **free()** (Deallocate Memory)

- **Purpose:** Frees previously allocated memory.

Syntax:

```
void free(void* ptr);
```

- **Description:** **free()** deallocates memory that was previously allocated using **malloc()**, **calloc()**, or **realloc()**. After calling **free()**, the pointer **ptr** is no longer valid, and using it can lead to **undefined behavior** (i.e., **dangling pointer**).
 - After freeing memory, the pointer is **not automatically set to NULL**. It's a good practice to set the pointer to **NULL** after freeing it to avoid accidental use.

Example:

```
int *arr = (int*) malloc(5 * sizeof(int));

// Use the allocated memory...

free(arr); // Deallocates memory

arr = NULL; // Avoid dangling pointer

#include <stdio.h>

#include <stdlib.h>

int main() {

    // Allocate memory using malloc

    int *arr = (int*) malloc(5 * sizeof(int));

    if (arr == NULL) {

        printf("Memory allocation failed.\n");

        return 1; // Exit if memory allocation fails

    }
```

```

// Initialize memory with values

for (int i = 0; i < 5; i++) {

    arr[i] = i + 1;

}

// Print the values

printf("Array values using malloc:\n");

for (int i = 0; i < 5; i++) {

    printf("%d ", arr[i]);

}

printf("\n");


// Reallocate memory using realloc to hold 10 integers

arr = (int*) realloc(arr, 10 * sizeof(int));

if (arr == NULL) {

    printf("Memory reallocation failed.\n");

    return 1;

}

// Initialize new memory elements

for (int i = 5; i < 10; i++) {

    arr[i] = i + 1;

}

// Print the values after realloc

printf("Array values after realloc:\n");

for (int i = 0; i < 10; i++) {

    printf("%d ", arr[i]);

}

printf("\n");

// Free the allocated memory

free(arr);

arr = NULL;

return 0;}

```

7c.

Define recursion. Develop a C program and a function to compute factorial of a given number using recursion.

Recursion is a programming technique where a function calls itself to solve a problem.

A recursive function typically contains:

1. Base Case – the condition under which the function stops calling itself.
2. Recursive Case – the part where the function calls itself with a smaller/simpler input.

Recursion is useful for solving problems that can be broken into smaller subproblems, such as factorial, Fibonacci series, tree traversal, etc.

```
#include <stdio.h>
```

```
// Recursive function to compute factorial
```

```
long long factorial(int n) {
```

```
    if (n == 0 || n == 1) { // Base case
```

```
        return 1;
```

```
    } else {
```

```
        return n * factorial(n - 1); // Recursive call
```

```
    }
```

```
}
```

```
int main() {
```

```
    int num;
```

```
    printf("Enter a number: ");
```

```
    scanf("%d", &num);
```

```
    if (num < 0) {
```

```
        printf("Factorial of a negative number doesn't exist.\n");
```

```
    } else {
```

```
        printf("Factorial of %d is %lld\n", num, factorial(num));
```

```
    }
```

8a.	<pre>return 0; }</pre> Output: Enter a number: 5 Factorial of 5 is 120 List the advantages of functions in programming. With suitable program, how pointer is initialized to a function for call/reference? Advantages of Functions in Programming 1. Modularity ○ Functions break a large program into smaller, manageable parts. 2. Reusability ○ A function can be used multiple times in the program without rewriting code. 3. Improved Readability ○ Code becomes easier to read, understand, and maintain. 4. Easy Debugging ○ Errors can be isolated inside specific functions, making debugging simpler. 5. Abstraction ○ The user only needs to know what a function does, not how it works internally. 6. Avoids Code Duplication ○ Instead of repeating a block of code many times, the function is written once and reused. 7. Helps in Teamwork ○ Different people can work on different functions of the same program. A function pointer stores the address of a function. It allows calling the function using the pointer, enabling: ● Callbacks ● Event-driven programming ● Flexible and dynamic function calling Syntax: <code>return_type (*pointer_name)(parameter_list) = function_name;</code>			
-----	--	--	--	--

Program: #include <stdio.h>

// A simple function

int add(int a, int b) {

 return a + b;

}

int main() {

 // Declare a function pointer

 int (*func_ptr)(int, int);

 // Initialize pointer to the function 'add'

 func_ptr = add;

 // Call the function using the pointer

 int result = func_ptr(10, 20);

 printf("The sum is: %d\n", result);

 return 0;

}

Output: The sum is: 30

8 b)

Explain TWO techniques of parameter passing to functions with suitable program segments.

Ans:

[6]

Two Techniques of Parameter Passing in C

1. Call by Value

Definition

In **call by value**, the function receives a **copy** of the actual argument.
Changes made inside the function **do NOT affect** the original variables.

✓ Program Segment (Call by Value)

```
#include <stdio.h>
```

```
void change(int x) {  
    x = x + 10; // changes local copy only  
    printf("Inside function (x): %d\n", x);  
}
```

```
int main() {  
    int a = 5;  
    change(a); // pass by value  
    printf("In main (a): %d\n", a); // original remains same  
    return 0;  
}
```

2. Call by Reference

Definition

In **call by reference**, the **address** of the variable is passed to the function.
Thus, changes made inside the function **directly modify** the original variable.

✓ Program Segment (Call by Reference)

```
#include <stdio.h>
```

```
void change(int *x) {  
    *x = *x + 10; // changes actual variable using pointer  
}
```

```
int main() {  
    int a = 5;  
    change(&a); // pass address of variable  
    printf("After function call (a): %d\n", a); // value modified  
    return 0;  
}
```

8 c)

Develop a C-program and a function to check whether the given number is prime or not.

[8]

Ans:

```
#include <stdio.h>

// Function to check prime
int isPrime(int n) {
    int i;

    if (n <= 1)
        return 0;    // 0 and 1 are not prime

    for (i = 2; i <= n/2; i++) {
        if (n % i == 0)
            return 0;    // not prime
    }
    return 1;    // prime
}

int main() {
    int num;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (isPrime(num))
        printf("%d is a Prime Number\n", num);
    else
        printf("%d is Not a Prime Number\n", num);

    return 0;
}
```

O/P:

```
PS C:\Users\91819\OneDrive\Desktop\C_Programming>
Enter a number: 13
13 is a Prime Number
PS C:\Users\91819\OneDrive\Desktop\C_Programming>
Enter a number: 90
90 is Not a Prime Number
PS C:\Users\91819\OneDrive\Desktop\C_Programming>
```

9 a)	Define a structure in C. Explain the different types of structure declarations with examples. Ans: A structure in C is a user-defined data type that allows storing a collection of variables of different data types under a single name. It is used to group related data items like student details, employee records, etc. Types of Structure Declarations in C 1. Normal Structure Declaration (Tag + Members) This is the most common method. Syntax <pre>struct structure_name { data_type member1; data_type member2; };</pre> 2. Structure Declaration With Variable Creation You can create structure variables at the time of declaring the structure. Example <pre>struct employee { int id; float salary; } e1, e2; // structure variables created</pre> 3. Anonymous Structure (No Tag Name) Structure has no name, only variables. Example: <pre>struct { char name[20]; int age; } person1, person2;</pre> Here, we cannot create more variables later because the structure has no tag name. 4. Structure With typedef Useful to create a short alias name for the structure. Example <pre>typedef struct person { char name[20]; int age; } P;</pre>	[6]	L2	CO4
------	---	-----	----	-----

P p1, p2; // using typedef alias

5. Nested Structure (Structure inside Structure)

A structure contains another structure as a member.

Example

```
struct address {  
    char city[20];  
    int pincode;  
};
```

```
struct student {  
    int roll;  
    struct address addr; // nested structure  
};
```

6. Array of Structures

Used to store multiple records of the same type.

Example

```
struct student {  
    int roll;  
    float marks;  
};
```

```
struct student s[50]; // array of 50 students
```

9 b)	Describe a method to compare two structure variables of the same type, and provide a simple example. Ans: Comparing Two Structure Variables in C C does not allow direct comparison of structures using <code>==</code>. Therefore, to compare two structure variables of the same type, we must compare each member individually. Method 1. Take two structure variables of the same structure type. 2. Compare each corresponding member using <code>==</code>. 3. If all members are equal → structures are equal. 4. If any member is not equal → structures are different. Example Program <pre>#include <stdio.h> #include <string.h> struct student { int roll; char name[20]; float marks; }; int main() { struct student s1 = {1, "Raj", 85.5}; struct student s2 = {1, "Raj", 85.5}; // Compare each member if (s1.roll == s2.roll && strcmp(s1.name, s2.name) == 0 && s1.marks == s2.marks) { printf("Both structures are equal.\n"); } else { printf("Structures are not equal.\n"); } return 0; }</pre>	[6]	L2	CO4
------	---	-----	----	-----

9 c)	Define a structure with a name student. Develop a C program that uses a structure named student. The program should read and display the details of 'N' students, compute the average marks of the class, and identify the students who have scored marks above and below the class average. Ans: <pre> #include <stdio.h> struct student { char name[50]; int roll; float marks; }; // Function prototypes void readStudents(struct student *s, int n); void displayStudents(struct student *s, int n); float computeAverage(struct student *s, int n); void aboveBelowAverage(struct student *s, int n, float avg); int main() { int n; printf("Enter number of students: "); scanf("%d", &n); struct student s[n]; readStudents(s, n); printf("\n--- Student Details ---\n"); displayStudents(s, n); float avg = computeAverage(s, n); printf("\nClass Average Marks = %.2f\n", avg); aboveBelowAverage(s, n, avg); return 0; } void readStudents(struct student *s, int n) { for(int i = 0; i < n; i++) { printf("\nEnter details of student %d\n", i + 1); printf("Name: "); scanf("%s", s[i].name); printf("Roll No: "); scanf("%d", &s[i].roll); </pre>	[8]	L3	[8]
------	--	-----	----	-----

```

        printf("Marks: ");
        scanf("%f", &s[i].marks);
    }
}

void displayStudents(struct student *s, int n) {
    for(int i = 0; i < n; i++) {
        printf("\nStudent %d\n", i + 1);
        printf("Name : %s\n", s[i].name);
        printf("Roll : %d\n", s[i].roll);
        printf("Marks: %.2f\n", s[i].marks);
    }
}

float computeAverage(struct student *s, int n) {
    float sum = 0;
    for(int i = 0; i < n; i++) {
        sum += s[i].marks;
    }
    return sum / n;
}

void aboveBelowAverage(struct student *s, int n, float avg) {
    printf("\nStudents scoring ABOVE average:\n");
    for(int i = 0; i < n; i++) {
        if(s[i].marks > avg)
            printf("%s (%.2f)\n", s[i].name, s[i].marks);
    }

    printf("\nStudents scoring BELOW average:\n");
    for(int i = 0; i < n; i++) {
        if(s[i].marks < avg)
            printf("%s (%.2f)\n", s[i].name, s[i].marks);
    }
}

```

O/P:

PS C:\Users\91819\OneDrive\Desktop\C_Programming> ./a.exe

Enter number of students: 3

Enter details of student 1

Name: rajni

Roll No: 101

Marks: 95

Enter details of student 2

Name: Anil

Roll No: 102

Marks: 87

Enter details of student 3

Name: mayra

Roll No: 103

Marks: 45

--- Student Details ---

Student 1

Name : rajni

Roll : 101

Marks: 95.00

Student 2

Name : Anil

Roll : 102

Marks: 87.00

Student 3

Name : mayra

Roll : 103

Marks: 45.00

Class Average Marks = 75.67

Students scoring ABOVE average:

rajni (95.00)

Anil (87.00)

Students scoring BELOW average:

mayra (45.00)

10a)	Compare the structure and union in terms of syntax, storage and uses/applicability. Ans: Difference Between Structure and Union (Exam Table) <table><tr><th>Point</th><th>Structure</th><th>Union</th></tr><tr><td>Definition</td><td>Collection of different data types grouped together.</td><td>Collection of different data types sharing the same memory.</td></tr><tr><td>Syntax</td><td>Declared using <code>struct</code>.</td><td>Declared using <code>union</code>.</td></tr><tr><td>Memory Allocation</td><td>Allocates separate memory for each member.</td><td>Allocates one shared memory for all members, the size of the largest member.</td></tr><tr><td>Total Size</td><td>Size = sum of all members (plus padding).</td><td>Size = largest member size only.</td></tr><tr><td>Access of Members</td><td>All members can store data simultaneously.</td><td>Only one member can store data at a time.</td></tr></table>	Point	Structure	Union	Definition	Collection of different data types grouped together.	Collection of different data types sharing the same memory.	Syntax	Declared using <code>struct</code> .	Declared using <code>union</code> .	Memory Allocation	Allocates separate memory for each member.	Allocates one shared memory for all members, the size of the largest member.	Total Size	Size = sum of all members (plus padding).	Size = largest member size only.	Access of Members	All members can store data simultaneously.	Only one member can store data at a time.	[6]	L2	CO4
Point	Structure	Union																				
Definition	Collection of different data types grouped together.	Collection of different data types sharing the same memory.																				
Syntax	Declared using <code>struct</code> .	Declared using <code>union</code> .																				
Memory Allocation	Allocates separate memory for each member.	Allocates one shared memory for all members, the size of the largest member.																				
Total Size	Size = sum of all members (plus padding).	Size = largest member size only.																				
Access of Members	All members can store data simultaneously.	Only one member can store data at a time.																				

10 b)	Define Enumerated data type. Explain the declaration and access of enumerated data types with the help of C program segment. Ans: Enumerated Data Type (enum) Definition An enumerated data type, known as enum, is a user-defined data type in C that consists of a set of named integer constants. It is used to assign symbolic names to integral values, which improves code readability and maintainability. Declaration of enum Syntax <pre>enum enum_name { value1, value2, value3, ... };</pre> - By default, the first name has value 0, the next 1, and so on. - You can also assign custom integer values. Example <pre>enum week { MON, TUE, WED, THU, FRI, SAT, SUN };</pre> Accessing enum Variables Steps Declare an enum type Create variables of that type Assign or use the enum values C Program: <pre>#include <stdio.h> enum week { MON = 1, TUE, WED, THU, FRI, SAT, SUN }; int main() { enum week today; today = WED; // accessing enum value printf("Numeric value of today is: %d\n", today); if (today == WED) { printf("It is Wednesday!\n"); } return 0; }</pre>	[6]	L2	CO4
-------	--	-----	----	-----

10 c)	Develop a C program to access and modify the members of structures, in array of structures in C. Ans: <pre> #include <stdio.h> struct student { int roll; char name[20]; float marks; }; int main() { int i, n; printf("Enter number of students: "); scanf("%d", &n); struct student s[n]; // array of structures // ----- Reading student details ----- printf("\nEnter details of students:\n"); for (i = 0; i < n; i++) { printf("\nStudent %d:\n", i + 1); printf("Enter Roll No: "); scanf("%d", &s[i].roll); printf("Enter Name: "); scanf("%s", s[i].name); printf("Enter Marks: "); scanf("%f", &s[i].marks); } // ----- Modifying the marks of a student ----- int r; float newMarks; printf("\nEnter roll number of student to update marks: "); scanf("%d", &r); printf("Enter new marks: "); scanf("%f", &newMarks); for (i = 0; i < n; i++) { if (s[i].roll == r) { s[i].marks = newMarks; // modifying value printf("Marks updated successfully!\n"); break; } } } </pre>	[8]	L3	CO4
-------	---	-----	----	-----

```
// ----- Displaying updated records -----  
printf("\nUpdated Student Records:\n");  
for (i = 0; i < n; i++) {  
    printf("\nRoll: %d\nName: %s\nMarks: %.2f\n",  
        s[i].roll, s[i].name, s[i].marks);  
}  
  
return 0;  
}
```

O/P:

PS C:\Users\91819\OneDrive\Desktop\C_Programming> .

Enter number of students: 3

Enter details of students:

Student 1:

Enter Roll No: 101

Enter Name: Rajni

Enter Marks: 85

Student 2:

Enter Roll No: 102

Enter Name: abc

Enter Marks: 94

Student 3:

Enter Roll No: 103

Enter Name: xyz

Enter Marks: 76

Enter roll number of student to update marks: 102 Enter new marks: 89 Marks updated successfully! Updated Student Records: Roll: 101 Name: Rajni Marks: 85.00 Roll: 102 Name: abc Marks: 89.00 Roll: 103 Name: xyz Marks: 76.00			
--	--	--	--